Cloud Optimization: TD1

Local Workloads, Linux Setup, and Benchmarking

Group 19: Yuba, Amani, and Hichem
A4, Cloud Computing & Cybersecurity (CCC)

Session: 2026-09-11 Report finalized: 2026-09-17

Abstract

TD1 required no cloud infrastructure. Its purpose was to give us a disciplined, reproducible local benchmarking methodology (environment characterization, compilation, timing, and a controlled measurement campaign) before we attempt any cross-machine or cross-VM comparison in later sessions. In this report we document that methodology as we applied it across our three laptops, and we present two findings that go beyond what the TD sheet asked for: a cache cliff in `matrix_task` and a $\sim 2\times$ runtime effect from CPU power state.

Contents

1 Objectives and scope 3

2 Environment 3

3 Compilation 4

4 The four workloads 4

5 Timing methodology 4

6 Benchmark procedure and local campaign 5

7 Finding 1: a cache cliff in `matrix_task` 6

8 Finding 2: power state moves every runtime by $\sim 2\times$ 7

9 Cross-machine comparison 7

10 Conclusion 8

1 Objectives and scope

Per the TD1 sheet, our learning objectives were to prepare a Linux environment, compile a Go program to a native executable, characterize our local machines, measure execution time and memory, run repeated experiments across several workload sizes, and produce a reproducible local benchmark dataset. TD1 is not graded and has no formal submission, so this report is our own record, as a group, of what we did and what we found, alongside the raw data in our project repository.

2 Environment

2.1 Our three machines

We ran the benchmarks independently on our own laptops so that we would have more than one data point from the start. Our primary campaign (Sections 6 to 8) uses Hichem's machine, `ultra9-285h`; Section 9 brings in Amani's and Yuba's machines for comparison.

	Hichem (`ultra9-285h`)	Amani	Yuba
Distribution	Ubuntu 24.04.4 LTS	(WSL2)	Debian 12 (bookworm)
Environment	WSL2	WSL2	WSL2
CPU	Core Ultra 9 285H	Core Ultra 7 268V	i7-11800H
Cores / threads	16 physical, 1/core	8 physical, 1/core	8 physical, 2/core (SMT)
Logical CPUs	16	8	16
L3 cache	24 MiB	12 MiB	24 MiB
RAM	19 GiB	(not recorded)	15 GiB
Go	go1.22.2	(see machine file)	go1.19.8

We are all on WSL2, which is a virtual machine on a Windows host rather than bare metal. We confirmed WSL2 is acceptable for this project, but we still think it is worth declaring explicitly: it is exactly the kind of environment difference the sheet warns us against ignoring when comparing results.

2.2 Why we recorded the environment first

A runtime is not a property of a program in isolation. It is a property of a program running on a specific machine, so without the machine recorded, a number cannot be interpreted or reproduced. This is also why every row of our benchmark CSV (Section 6) carries a machine label rather than being pooled anonymously.

2.3 Core vs. logical CPU

A core is a physical execution unit; a logical CPU is what the OS scheduler can dispatch onto. They differ only when simultaneous multithreading (SMT, or hyperthreading) lets one core present as two scheduling targets. Hichem's and Amani's machines both report one thread per core, so core count equals logical CPU count for us on those two. Yuba's machine is the exception (Section 9): 8 physical cores but 16 logical CPUs, with SMT on.

3 Compilation

We compiled the four reference workloads (`cpu_task`, `memory_task`, `matrix_task`, `sort_task`) with:

```
go build -o bin/cpu_task    workloads/cpu_task.go
go build -o bin/memory_task workloads/memory_task.go
go build -o bin/matrix_task workloads/matrix_task.go
go build -o bin/sort_task   workloads/sort_task.go
```

`go build` produces one self-contained native executable per source file (∼2.3 MB, `ELF 64-bit LSB executable, x86-64, statically linked`; `ldd` reports *not a dynamic executable*). Because Go compiles ahead of time, nothing is translated or interpreted inside a timed region, so our measurements reflect the work itself, not a runtime. We confirmed the binaries run standalone, with source and environment stripped:

```
$ env -i ./cpu_task --size 100000
{"task":"cpu_task","size":100000,"runtime_s":0.007179392,"primes_found":9592}
```

4 The four workloads

Workload	What it does	`--size` means	Cost shape
`cpu_task`	counts primes by trial division	largest integer tested	$n^{1.5}$
`memory_task`	allocates + sweeps an array (5 passes)	megabytes allocated	linear
`matrix_task`	naive dense $N \times N$ multiply	matrix dimension N	N^3
`sort_task`	generates then sorts an integer array	number of integers	$n \log n$

We found `cpu_task` to be the most CPU-intensive workload: a flat 2 MB footprint at every size, and 99% user time (4.55 s user versus 0.05 s system at its largest tested size). `memory_task` uses the most memory: peak RSS tracks the requested size almost exactly (1024 MB requested, 1027 MB resident). Taking size as a command-line parameter let us produce a controlled range of work from one binary, which is what makes it possible to measure how cost grows with input rather than only at one point.

Two implementation details we want to flag for whoever automates this in TD3: `memory_task` emits a `size_mb` field where the other three emit `size`, and the four programs' internal timers start at different points in their own execution (Section 5).

5 Timing methodology

5.1 Wall-clock vs. CPU time

Wall-clock time is elapsed real time. CPU time is time a core actually spent on the process, split into user (its own instructions) and system (kernel work on its behalf); the two diverge whenever the process is not actively running. `memory_task` is the one workload where we measured system time exceeding user time: the real cost is the kernel supplying and zeroing a gigabyte of pages, not the program touching it. It is also the only workload that drops below 100% CPU, blocking on page faults.

5.2 What we decided about `runtime_s`

We settled two measurement questions ourselves, directly from the sheet's own text and our own data, without needing to wait on an external answer.

- **We used the GNU `time` wall clock, not each program's internal JSON figure.** The sheet defines `runtime_s` as elapsed time from start to end of execution, and the external wall clock is the only definition consistent across all four workloads. The internal timers disagree with wall clock by workload: about 1% for `cpu_task` and `memory_task`, about 1% for `matrix_task` (which excludes filling its input matrices), and 10% for `sort_task` (which excludes generating its input array). Comparing internal figures across workloads would have silently compared different things, so we kept both: the CSV holds wall clock, and we preserved the per-run JSON under `data/raw/`.
- **We treated one CSV row as one single execution, never a mean.** The sheet's own Part 6 instructions are explicit: keep the individual measurements, and "compute the average runtime only during the analysis phase." Our CSV schema (`workload,size,run,runtime_s,max_rss_kb`) has a dedicated `run` column precisely so raw repetitions are never collapsed on write.

5.3 Why two runs of the same program differ

We identified four causes, largest effect first:

1. **CPU power state, $\sim 2\times$.** The single largest effect we measured in this TD; see Finding 2, Section 8.
2. **The definitional gap between internal and wall-clock timers** (not true variation), quantified above and largest for `sort_task`.
3. **Cache effects.** Small between repeats at fixed size, large across sizes; see Finding 1, Section 7.
4. **Scheduling and other processes.** Repeats within our campaign spread 1 to 5%.

6 Benchmark procedure and local campaign

We wrote a Bash runner, `scripts/benchmark.sh`, to automate the campaign: it selects a workload and size, runs it a configurable number of times, measures wall clock and peak RSS via `/usr/bin/time -v`, appends one CSV row per execution, and reports a clear error (exit non-zero, no row written) if the binary is missing or a run fails. Every row carries the producing machine's label, matching a `data/machine_<label>.txt` spec file.

The sheet's suggested sizes all completed in under 0.25 s on Hichem's CPU, several in under 0.01 s, indistinguishable from process startup and scheduler noise. We scaled the sizes up until the largest run took roughly 2 s:

Workload	Sheet suggested	Used here	Largest run
`cpu_task`	200k / 500k / 1M	2.5M / 5M / 10M	2.12 s
`memory_task`	128 / 256 / 512 MB	256 / 512 / 1024 MB	1.13 s
`matrix_task`	200 / 400 / 600	500 / 1000 / 1500	4.50 s
`sort_task`	(absent from the sheet's campaign)	5M / 10M / 20M	1.87 s

We left `--passes` at its default of 5 throughout, since changing it would make sizes incomparable. The results below are the mean of 3 runs, measured on mains power, across the 36 rows in `data/benchmark_local.csv`.

Workload	Size	Mean (s)	Min	Max	Peak RSS
`cpu_task`	2 500 000	0.283	0.28	0.29	2 MB
`cpu_task`	5 000 000	0.750	0.75	0.75	2 MB
`cpu_task`	10 000 000	2.120	2.02	2.22	2 MB
`memory_task`	256	0.300	0.27	0.36	258 MB
`memory_task`	512	0.547	0.49	0.63	514 MB
`memory_task`	1024	1.127	1.05	1.28	1027 MB
`matrix_task`	500	0.073	0.07	0.08	8 MB
`matrix_task`	1000	0.550	0.54	0.56	25 MB
`matrix_task`	1500	4.503	4.13	4.73	54 MB
`sort_task`	5 000 000	0.423	0.41	0.43	40 MB
`sort_task`	10 000 000	0.863	0.85	0.89	79 MB
`sort_task`	20 000 000	1.870	1.83	1.91	155 MB

7 Finding 1: a cache cliff in `matrix_task`

We predicted scaling from each workload's cost shape before measuring. Doubling the input does *not* double the runtime, except for the linear workload:

Workload	Step	Predicted	Observed	
`cpu_task`	5M → 10M	2.83×	2.83×	matches
`memory_task`	512 → 1024	2×	2.06×	matches
`sort_task`	10M → 20M	∼2.1×	2.17×	matches
`matrix_task`	500 → 1000	8×	7.53×	matches
`matrix_task`	**1000 → 1500**	3.38×	8.19×	**2.4× too slow**

`matrix_task` holds three dense $N \times N$ float64 matrices, so its working set is $3N^2 \times 8$ bytes:

N	Predicted working set	vs. 24 MiB L3	Measured RSS
500	6 MB	fits	8 MB
1000	24 MB	at the boundary	25 MB
1500	54 MB	2.25× over	54 MB

Once the working set leaves last-level cache, inner-loop steps become DRAM round-trips, which explains the jump from 3.38× to 8.19×. Our measured RSS matches the predicted working set at all three sizes, and the $N = 1500$ run also has the widest spread in our campaign (4.13 to 4.73 s, about 14%) against about 5% for `cpu_task`: the signature of a memory-bound run.

We consider this the finding that matters most for our project. Three of the four workloads can be ranked across machine types from clock speed alone. `matrix_task` cannot: its cost depends on cache hierarchy, which is invisible in a vCPU count or a price list. Any scheduler that models task cost as a single scalar per machine type will mispredict it.

8 Finding 2: power state moves every runtime by $\sim 2\times$

We re-measured roughly 90 minutes after our main campaign, on battery instead of mains, and got $\sim 2\times$ longer runtimes on an otherwise idle machine.

	Mains	Battery	Slowdown
`cpu_task` 10M	2.120	4.257	2.01×
`matrix_task` 1000 (fits L3)	0.550	1.367	2.49×
`matrix_task` 1500 (exceeds L3)	4.503	5.733	1.27×

Three repeats each. Power state is the largest single source of variation we measured in this TD, and we did not record it during the main campaign, so that baseline is not strictly reproducible as it stands. We also noticed, though we have not confirmed it, that the memory-bound case seems less sensitive to CPU throttling than the compute-bound ones, which would be consistent with DRAM speed not being governed by CPU power state. Confirming that would need a controlled comparison with power state pinned, which we have not done yet.

9 Cross-machine comparison

We collected data from Amani's and Yuba's machines independently and merged it into this report. The raw files live in `data/` alongside `ultra9-285h`'s.

9.1 Normalized `cpu_task` throughput

Amani and Yuba both benchmarked at sizes close to the sheet's own suggested values rather than the scaled-up sizes we used on `ultra9-285h`. Amani's `cpu_task` at size 200,000 still measured 0.01 s, the same sub-0.25 s unreliability we flagged in Section 6, now confirmed on a second machine. Dividing runtime by the $n^{1.5}$ cost shape lets us compare directly across the different sizes each of us used:

Machine	Size	Runtime	$t/n^{1.5}$
Hichem (16c)	10,000,000	2.120 s	6.70×10^{-11}
Yuba (8c/16t)	5,000,000	0.780 s	6.98×10^{-11}
Amani (8c)	1,000,000	0.09 s	9.00×10^{-11}

Hichem's and Yuba's machines land within about 4% of each other; Amani's machine is about 30 to 34% slower per unit of work. Since `cpu_task` is single-threaded, this isolates per-core speed, which we think plausibly reflects the 268V being a lower-power ("V-series") chip against the 285H's and 11800H's higher-power designs, rather than a core-count effect. We have not confirmed this against official specifications.

9.2 Cache cliff, extended across our machines

Machine	L3	Largest N tested	Predicted working set	vs. L3
Hichem	24 MiB	1500	54 MB	2.25× over: cliff observed
Amani	12 MiB	600	8.24 MB	fits (69% of L3): no cliff expected yet
Yuba	24 MiB	1000	24 MB	at the boundary: cliff plausible

Amani's predicted cliff, using her 12 MiB L3, is around $N \approx 707$. We have not tested that in her campaign, so we treat it as a prediction for a follow-up run rather than a confirmed result. Yuba's only tested size ($N = 1000$) sits almost exactly at his measured 24 MiB L3 boundary, the same boundary where we first saw the cliff on Hichem's machine in Finding 1. Yuba's `matrix_task` $N = 1000$ mean is 0.947 s against Hichem's 0.550 s at the same N, which we think is plausibly a mix of the general throughput gap above and sitting right at his cache boundary rather than comfortably inside it: Hichem had 6.5 MB of L3 headroom at $N = 1000$, while Yuba has essentially none. A run at $N = 1500$ on Yuba's machine would confirm whether his cliff lands where we predict.

9.3 `memory_task` across our machines

This one was harder for us to compare cleanly. Amani's first repetition at each size ran noticeably slower than her second and third (for example, size 512: 1.15 s, then 0.70 s, 0.66 s), a first-run effect we also see more weakly in Hichem's own data. Using the full three-run mean per the sheet's procedure, we get Hichem at about 0.0011 s/MB, Yuba at about 0.00072 s/MB, and Amani at about 0.00163 s/MB (about 0.00133 s/MB even discounting her outlier first run). Amani's machine looks slower here too, but the effect is noisier than `cpu_task`'s, so we do not treat it as confirmed. It would need repeats with a stable, recorded power state (Finding 2) to separate machine speed from run-to-run variance.

10 Conclusion

As a group, we found that the four workloads stress four different bottlenecks, confirmed by measurement rather than assumed from the sheet's labels. `cpu_task` is pure compute, scaling almost exactly with its $n^{1.5}$ cost shape. `memory_task` is allocation-bound, not compute-bound: system time exceeds user time because the kernel is supplying and zeroing pages. `matrix_task` looks CPU-bound at small sizes but is not: once its working set exceeds L3, doubling the input costs 8.19× instead of the predicted 3.38×. `sort_task` is the mixed case, with the largest gap (10%) between its internal timer and wall clock, which taught us to trust the external GNU `time` figure over each program's self-reported number.

The single biggest lesson we take from this TD is not about any one workload: power state moved every runtime by ∼2×, dwarfing the differences between workloads at fixed input size. Extending our comparison to three machines adds a second lesson: raw runtime is not comparable across machines unless normalized by input size against the workload's known cost shape, or Amani's faster-looking `cpu_task` numbers would be misleading. We believe any later VM comparison will be meaningless unless we record power state and machine specification alongside every measurement, exactly the discipline this TD asked of us from the start.